

Bài tập thực hành tuần 9

Họ và tên: Nguyễn Văn Phúc Huy

MSSV: 23110163

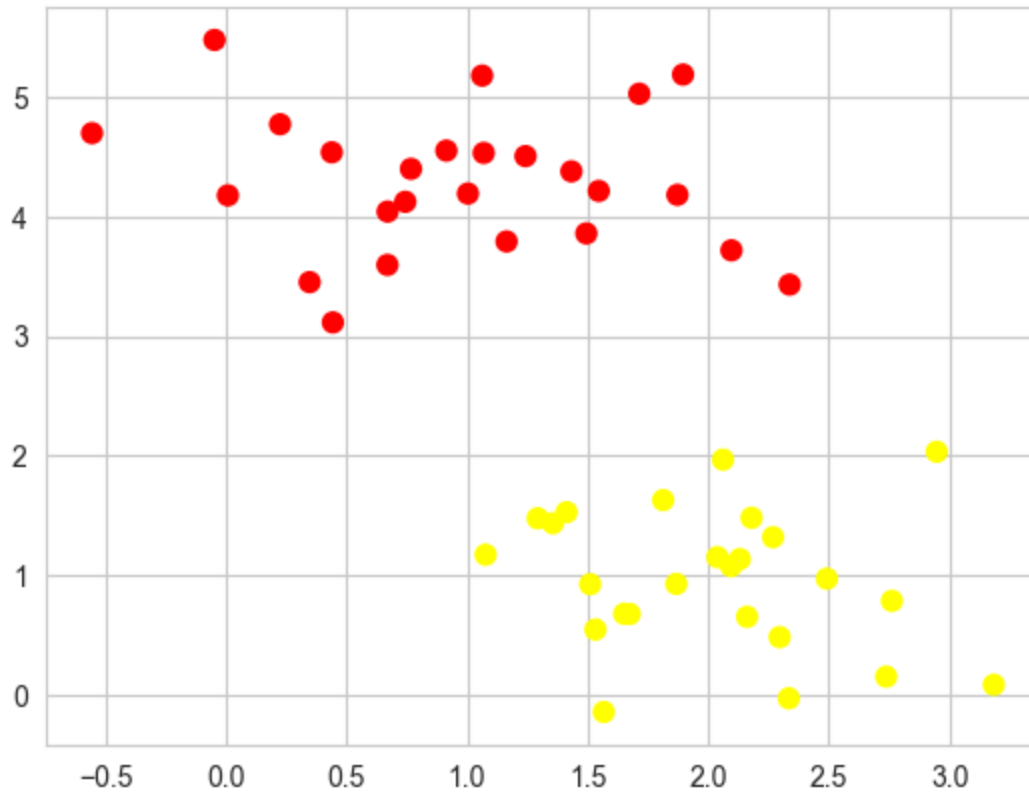
Lớp: 23TTH (Chiều thứ 6, ca 1)

```
In [5]: import random
import numpy as np
import pandas as pd
import seaborn as sns
import scipy.cluster.hierarchy as sch
import matplotlib.pyplot as plt
from sklearn.cluster import AgglomerativeClustering
from sklearn.mixture import GaussianMixture
from sklearn.datasets import make_blobs, make_circles, make_moons
from sklearn.cluster import KMeans
from sklearn.svm import SVC
sns.set_style('whitegrid')
from scipy import stats
from ipywidgets import interact, fixed
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

SVM với tập dữ liệu tách rời tuyến tính

Khởi tạo tập dữ liệu có 50 điểm và 2 cụm

```
In [6]: X, y = make_blobs(n_samples=50, centers=2, random_state=0, cluster_std=0.60)
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
plt.show()
```

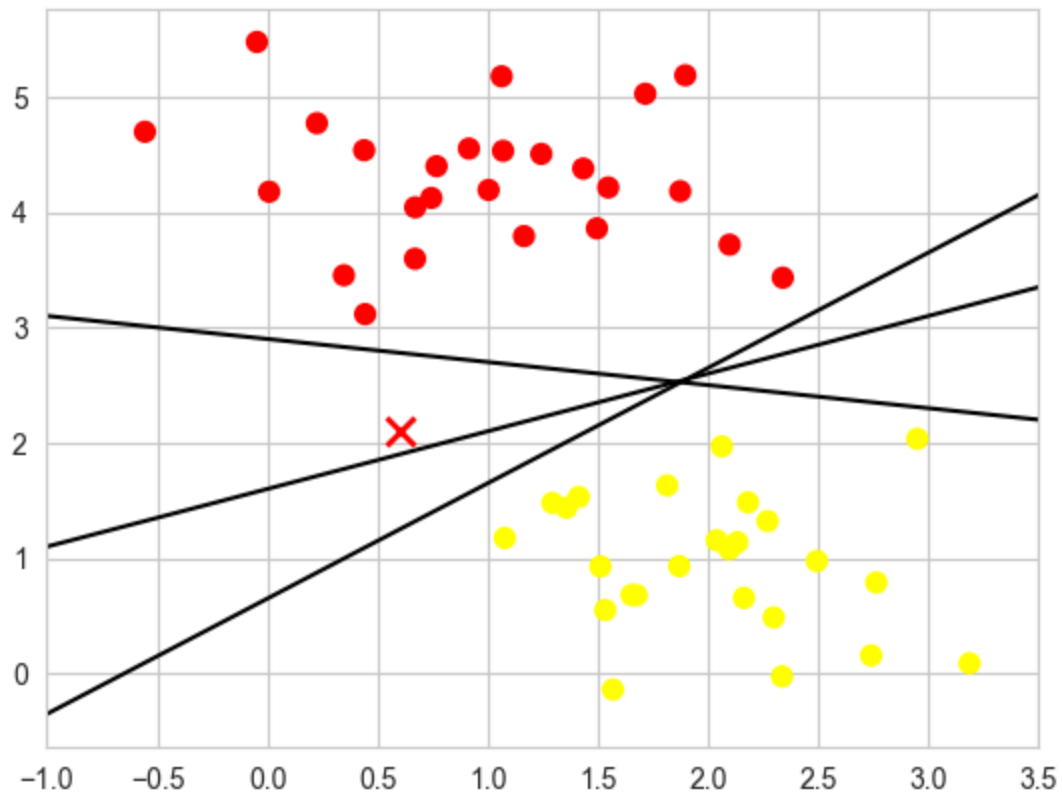


Vẽ đường thẳng phân tách 2 cụm

```
In [7]: xfit = np.linspace(-1, 3.5)
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
plt.plot([0.6], [2.1], 'x', color='red', markeredgewidth=2, markersize=10)

for m, b in [(1, 0.65), (0.5, 1.6), (-0.2, 2.9)]:
    plt.plot(xfit, m * xfit + b, '-k')

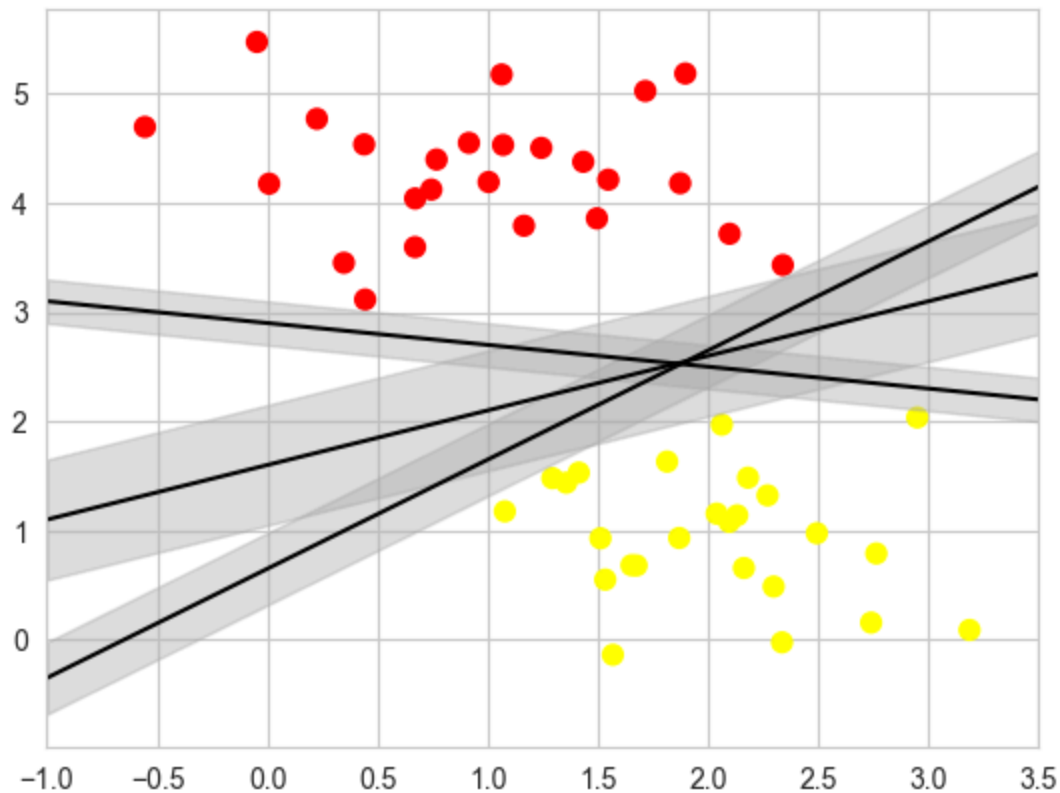
plt.xlim(-1, 3.5)
plt.show()
```



Vẽ siêu phẳng lề lớn nhất

```
In [8]: xfit = np.linspace(-1, 3.5)
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
for m, b, d in [(1, 0.65, 0.33), (0.5, 1.6, 0.55), (-0.2, 2.9, 0.2)]:
    yfit = m * xfit + b
    plt.plot(xfit, yfit, '-k')
    plt.fill_between(xfit, yfit - d, yfit + d, edgecolor='none', color='#AAAAAA', a

plt.xlim(-1, 3.5)
plt.show()
```



Sử dụng support vector classifier (SVC) của Scikit-Learn để huấn luyện mô hình SVM và vẽ các biên quyết định của SVM

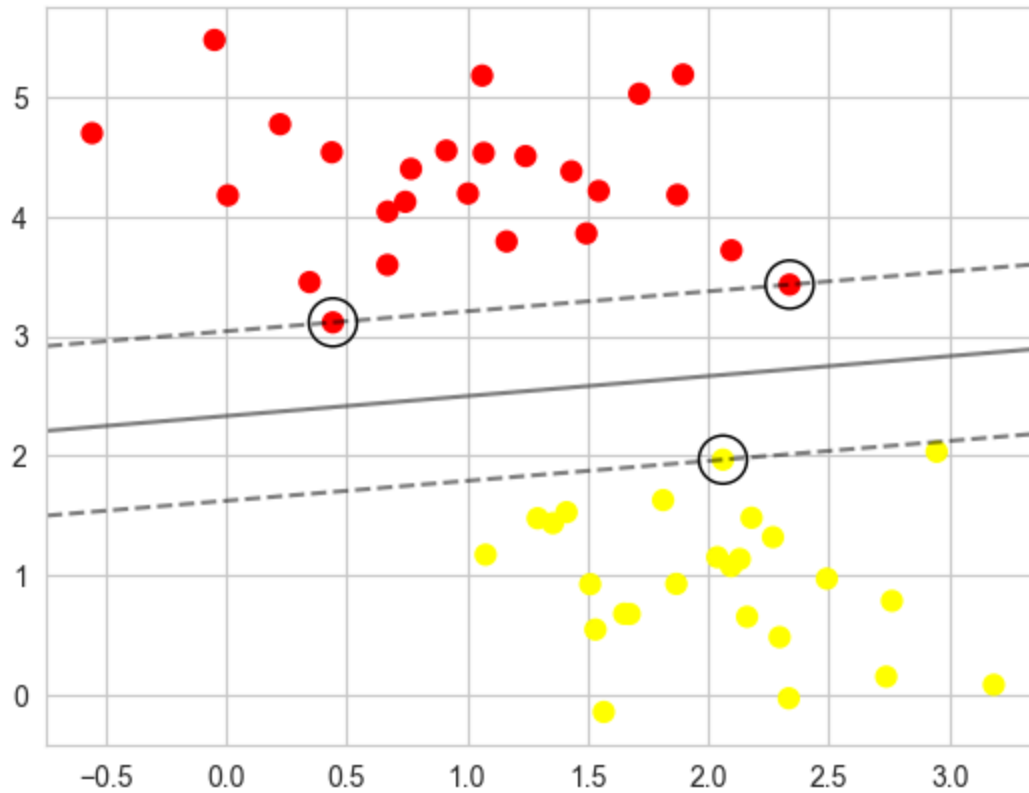
```
In [9]: model = SVC(kernel='linear', C=1E10)
model.fit(X, y)
def plot_svc_decision_function(model, ax=None, plot_support=True):
    if ax is None:
        ax = plt.gca()
    xlim = ax.get_xlim()
    ylim = ax.get_ylim()

    x = np.linspace(xlim[0], xlim[1], 30)
    y = np.linspace(ylim[0], ylim[1], 30)
    Y, X = np.meshgrid(y, x)
    xy = np.vstack([X.ravel(), Y.ravel()]).T
    P = model.decision_function(xy).reshape(X.shape)

    ax.contour(X, Y, P, colors='k', levels=[-1, 0, 1], alpha=0.5,
               linestyles=['--', '-', '--'])

    if plot_support:
        ax.scatter(model.support_vectors_[:, 0], model.support_vectors_[:, 1],
                  s=300, linewidth=1, facecolors='none', edgecolors='k')
    ax.set_xlim(xlim)
    ax.set_ylim(ylim)

plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
plot_svc_decision_function(model)
plt.show()
model.support_vectors_
```

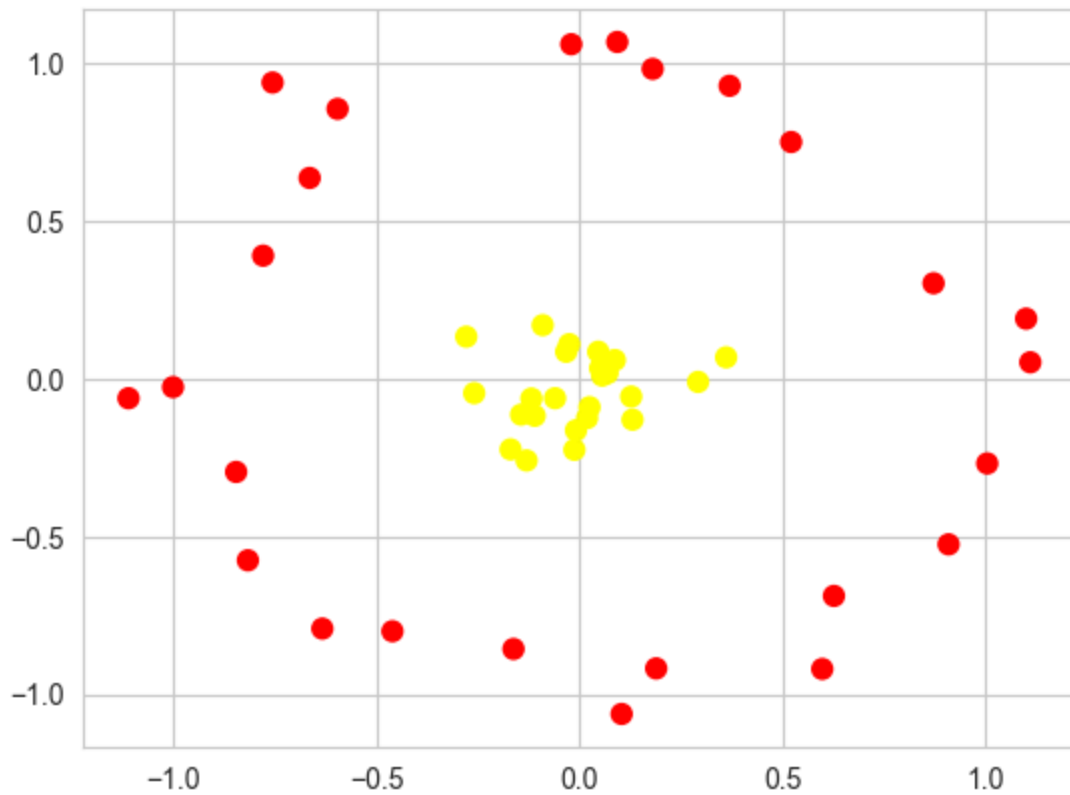


```
Out[9]: array([[0.44359863, 3.11530945],
               [2.33812285, 3.43116792],
               [2.06156753, 1.96918596]])
```

SVM với tập dữ liệu không tuyến tính

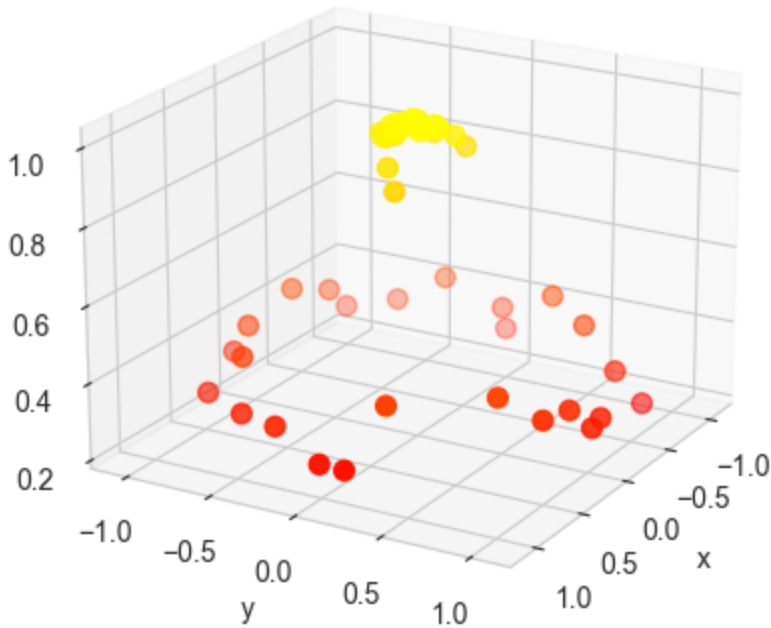
Khởi tạo tập dữ liệu có 50 điểm

```
In [10]: X, y = make_circles(50, factor = .1, noise = 0.1)
plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
plt.show()
```



Sử dụng hàm radial basis function và vẽ dữ liệu trong không gian 3 chiều

```
In [11]: r = np.exp(-(X ** 2).sum(1))
ax = plt.subplot(projection='3d')
ax.scatter3D(X[:, 0], X[:, 1], r, c=r, s=50, cmap='autumn')
ax.view_init(elev=20, azim=30)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('r')
plt.show()
```



Sử dụng support vector classifier (SVC) của Scikit-Learn để huấn luyện mô hình SVM và vẽ các biên quyết định của SVM

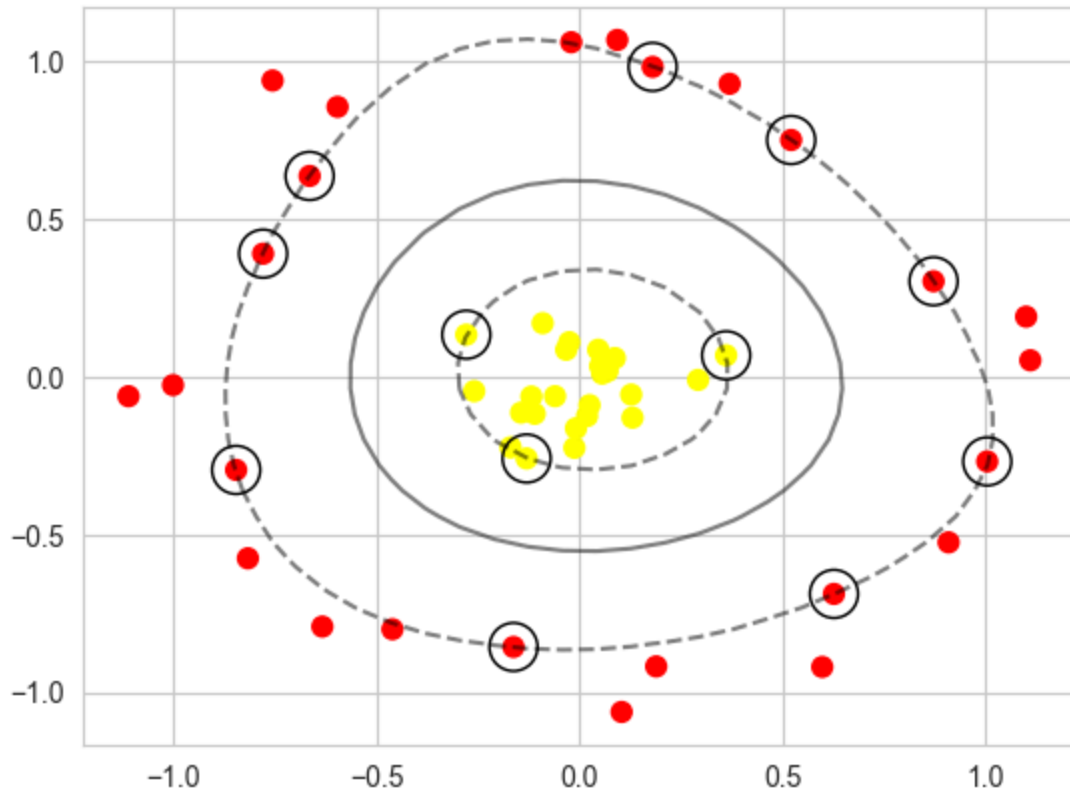
```
In [12]: model = SVC(kernel='rbf', C=1E6)
model.fit(X, y)
def plot_svc_decision_function(model, ax=None, plot_support=True):
    if ax is None:
        ax = plt.gca()
    xlim = ax.get_xlim()
    ylim = ax.get_ylim()

    x = np.linspace(xlim[0], xlim[1], 30)
    y = np.linspace(ylim[0], ylim[1], 30)
    Y, X = np.meshgrid(y, x)
    xy = np.vstack([X.ravel(), Y.ravel()]).T
    P = model.decision_function(xy).reshape(X.shape)

    ax.contour(X, Y, P, colors='k', levels=[-1, 0, 1], alpha=0.5,
               linestyles=['--', '-', '--'])

    if plot_support:
        ax.scatter(model.support_vectors_[:, 0], model.support_vectors_[:, 1],
                   s=300, linewidth=1, facecolors='none', edgecolors='k')
    ax.set_xlim(xlim)
    ax.set_ylim(ylim)

plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='autumn')
plot_svc_decision_function(model)
plt.show()
model.support_vectors_
```



```
Out[12]: array([[ 0.62764917, -0.68825631],
 [ 1.00554504, -0.26828226],
 [-0.66498755,  0.63680074],
 [ 0.87322223,  0.30293878],
 [-0.8462068 , -0.29518508],
 [ 0.18066449,  0.98254254],
 [ 0.52212463,  0.75074293],
 [-0.16241969, -0.85643361],
 [-0.77978477,  0.39040117],
 [-0.27877436,  0.13360838],
 [-0.13006067, -0.25884467],
 [ 0.36186446,  0.06808468]])
```

Áp dụng với tập dữ liệu banknote authentication từ UCI Machine Learning Repository. Kiểm tra banknote là tuyến tính hay không tuyến tính rồi áp dụng tương tự trường hợp tương ứng như trên.

```
In [13]: link_hd_ko_che= r'https://archive.ics.uci.edu/ml/machine-learning-databases/00267/d
columns = ['variance', 'skewness', 'curtosis', 'entropy', 'class']
df = pd.read_csv(link_hd_ko_che, names=columns)
df
```


Out[13]:

	variance	skewness	curtosis	entropy	class
0	3.62160	8.66610	-2.8073	-0.44699	0
1	4.54590	8.16740	-2.4586	-1.46210	0
2	3.86600	-2.63830	1.9242	0.10645	0
3	3.45660	9.52280	-4.0112	-3.59440	0
4	0.32924	-4.45520	4.5718	-0.98880	0
...	...	...	...	...	...
1367	0.40614	1.34920	-1.4501	-0.55949	1
1368	-1.38870	-4.87730	6.4774	0.34179	1
1369	-3.75030	-13.45860	17.5932	-2.77710	1
1370	-3.56370	-8.38270	12.3930	-1.28230	1
1371	-2.54190	-0.65804	2.6842	1.19520	1

1372 rows × 5 columns

Pre-processing data

```
In [14]: from sklearn.preprocessing import StandardScaler
X = df.drop('class', axis=1)
y = df['class']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Splitting data

```
In [15]: from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3, ran
```

```
In [16]: # Mô hình Tuyến tính (Linear Margin)
svm_linear = SVC(kernel='linear', C=1.0)
svm_linear.fit(X_train, y_train)
acc_linear = accuracy_score(y_test, svm_linear.predict(X_test))

# Mô hình Phi tuyến tính (Soft Margin - RBF Kernel)
svm_rbf = SVC(kernel='rbf', C=1.0, gamma='auto')
svm_rbf.fit(X_train, y_train)
acc_rbf = accuracy_score(y_test, svm_rbf.predict(X_test))

print(f"Độ chính xác của SVM Tuyến tính (Linear): {acc_linear * 100:.2f}%")
print(f"Độ chính xác của SVM Phi tuyến tính (RBF): {acc_rbf * 100:.2f}%")
```

Độ chính xác của SVM Tuyến tính (Linear): 98.79%

Độ chính xác của SVM Phi tuyến tính (RBF): 100.00%

```
In [17]: X_2d = X_scaled[:, :2]
```

```
In [19]: # Hàm vẽ biên quyết định (tương tự như trong bài thực hành)
def plot_svc_decision_function(model, ax=None, plot_support=True):
    if ax is None:
        ax = plt.gca()
    xlim = ax.get_xlim()
    ylim = ax.get_ylim()

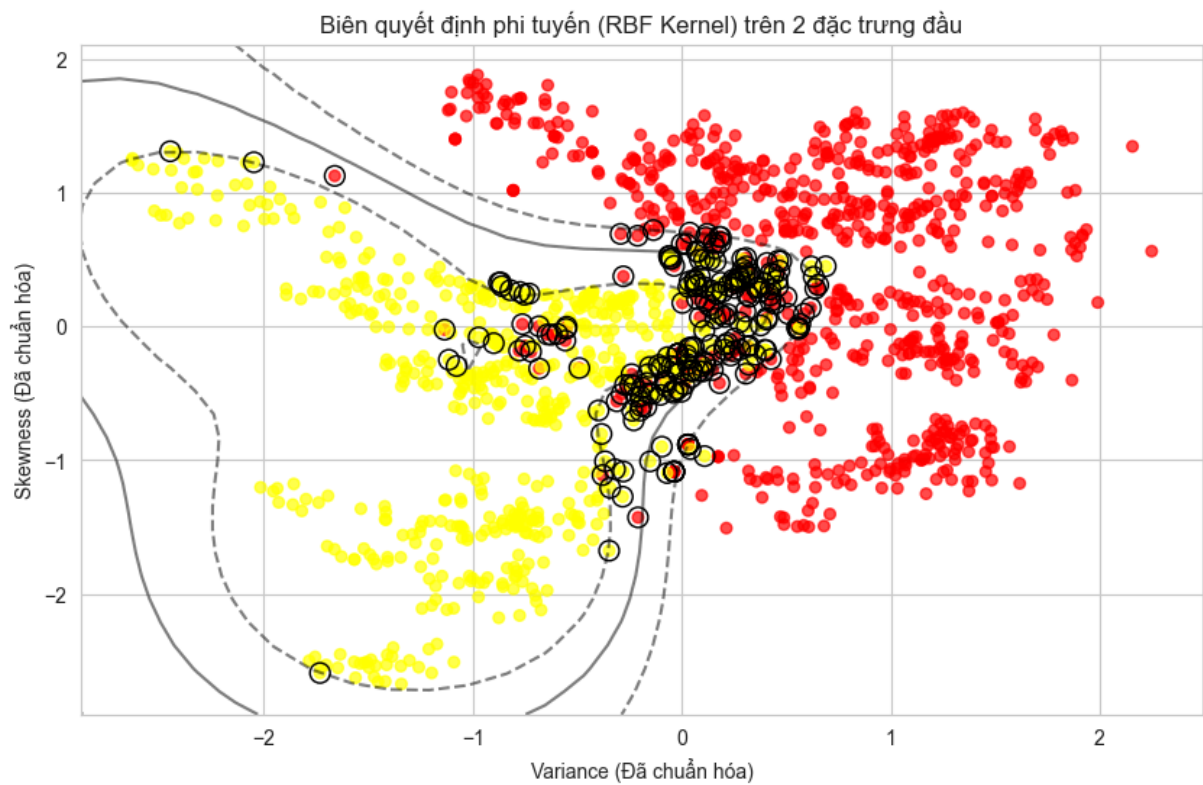
    # Tạo lưới để đánh giá mô hình
    x = np.linspace(xlim[0], xlim[1], 30)
    y_grid = np.linspace(ylim[0], ylim[1], 30)
    Y, X_grid = np.meshgrid(y_grid, x)
    xy = np.vstack([X_grid.ravel(), Y.ravel()]).T
    P = model.decision_function(xy).reshape(X_grid.shape)

    # Vẽ biên quyết định và Lề
    ax.contour(X_grid, Y, P, colors='k',
               levels=[-1, 0, 1], alpha=0.5,
               linestyles=['--', '-', '--'])

    # Vẽ support vectors
    if plot_support:
        ax.scatter(model.support_vectors_[0],
                  model.support_vectors_[1],
                  s=100, linewidth=1, edgecolors='black',
                  facecolors='none')
    ax.set_xlim(xlim)
    ax.set_ylim(ylim)

    # Huấn Luyện Lại mô hình RBF trên dữ liệu 2D để trực quan hóa
    svm_rbf_2d = SVC(kernel='rbf', C=10)
    svm_rbf_2d.fit(X_2d, y)

    # Vẽ đồ thị
    plt.figure(figsize=(10, 6))
    plt.scatter(X_2d[:, 0], X_2d[:, 1], c=y, s=30, cmap='autumn', alpha=0.7)
    plot_svc_decision_function(svm_rbf_2d)
    plt.title("Biên quyết định phi tuyến (RBF Kernel) trên 2 đặc trưng đầu")
    plt.xlabel("Variance (Đã chuẩn hóa)")
    plt.ylabel("Skewness (Đã chuẩn hóa)")
    plt.show()
    model.support_vectors_
```



```
Out[19]: array([[ 0.62764917, -0.68825631],  
               [ 1.00554504, -0.26828226],  
               [-0.66498755,  0.63680074],  
               [ 0.87322223,  0.30293878],  
               [-0.8462068 , -0.29518508],  
               [ 0.18066449,  0.98254254],  
               [ 0.52212463,  0.75074293],  
               [-0.16241969, -0.85643361],  
               [-0.77978477,  0.39040117],  
               [-0.27877436,  0.13360838],  
               [-0.13006067, -0.25884467],  
               [ 0.36186446,  0.06808468]])
```